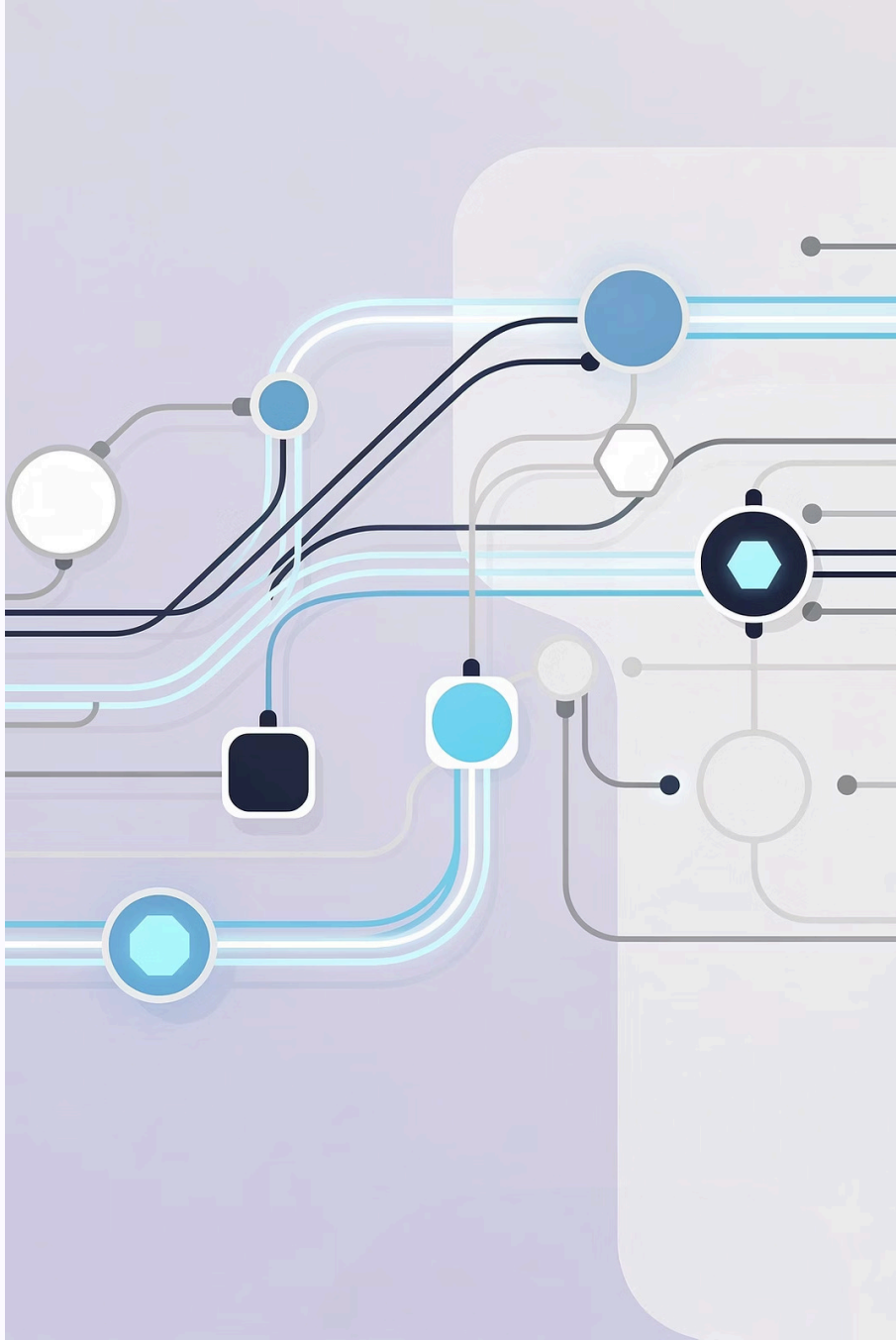
Delta Live Tables (DLT)

A modern, declarative approach to building reliable, maintainable data pipelines — directly within the Databricks Lakehouse Platform. Stop wiring pipelines by hand and start defining what your data should look like.

DATABRICKS | DATA ENGINEERING | ETL MODERNISATION



The Problem with Traditional Pipelines

Building and maintaining data pipelines the old way introduces compounding technical debt. Teams spend more time firefighting than delivering value.

Manual Orchestration

Engineers hand-wire task dependencies using tools like Airflow or cron jobs. Any change in upstream logic ripples downstream, requiring manual re-wiring, testing, and re-deployment — a fragile, time-consuming process.

Hard to Maintain

Pipeline logic is scattered across scripts, notebooks, and orchestration configs. Onboarding new engineers is slow, debugging failures is painful, and refactoring is risky without comprehensive test coverage.

No Data Quality Checks

Without built-in validation, bad data silently propagates through the pipeline. Downstream consumers — dashboards, ML models, reports — operate on corrupted or incomplete data, eroding trust in the platform.

What is Delta Live Tables?

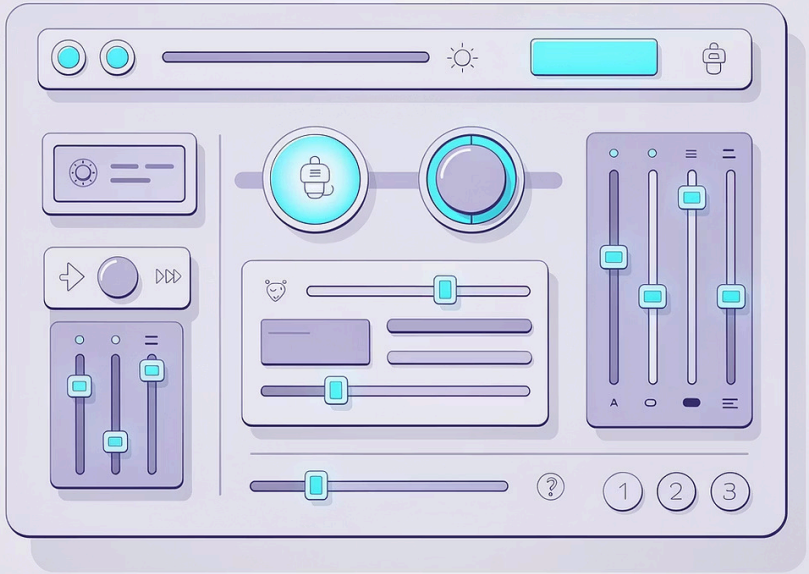
Delta Live Tables (DLT) is Databricks' framework for building **declarative, self-managing data pipelines**. Rather than coding how data moves, you simply define what each table should contain — and DLT handles the rest.

Declarative Pipelines

You define the desired state of each dataset using Python or SQL. DLT infers dependencies between tables automatically, builds the execution graph, and manages re-runs — no manual DAG construction required.

Managed Execution

DLT takes ownership of infrastructure management: cluster sizing, retries on failure, incremental vs. full refresh decisions, and lineage tracking. Your team focuses on transformation logic, not operational overhead.



Key Features of DLT

Auto Orchestration

DLT automatically resolves table dependencies and determines execution order. Add a new table and it slots into the pipeline — no scheduler config required.

Built-in Data Quality

Define expectations as constraints on your data. DLT enforces them at runtime, quarantining or dropping invalid records and surfacing quality metrics in the pipeline UI.

Incremental Processing

DLT uses Delta Lake's change data capture to process only new or changed records by default — dramatically reducing compute costs and latency on large datasets.

The Medallion Architecture

DLT is purpose-built to implement the Medallion Architecture — a layered data design pattern that progressively refines raw data into trusted, business-ready assets.

🥉 Bronze — Raw

Ingest raw data as-is from source systems. No transformations. This layer preserves the full historical record and acts as the system of record for re-processing.

🥈 Silver — Cleaned

Apply deduplication, type casting, null handling, and data quality expectations. This layer produces a reliable, conformed dataset suitable for general analytics use.

🥇 Gold — Aggregated

Business-level aggregations and domain-specific metrics. Optimised for BI tools, dashboards, and ML feature stores. Low latency, high trust.

DLT Pipeline: A Practical Example

Below is a minimal but complete DLT pipeline in Python. Each function decorated with `@dlt.table` defines a managed Delta table. DLT infers the dependency between `silver_table` and `bronze_table` automatically — no explicit wiring needed.

```python
import dlt
from pyspark.sql.functions import col

# --- Bronze Layer: Raw ingestion ---
@dlt.table(
  name="bronze_table",
  comment="Raw events ingested from the source system"
)
def bronze_table():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "json")
            .load("/mnt/raw/events/")
    )

# --- Silver Layer: Cleaned and validated ---
@dlt.table(
  name="silver_table",
  comment="Cleaned events with quality constraints applied"
)
@dlt.expect_or_drop("valid_amount", "amount > 0")
def silver_table():
    return (
        dlt.read_stream("bronze_table")
          .filter(col("event_type").isNotNull())
          .withColumnRenamed("ts", "event_timestamp")
    )
```

DLT resolves the execution order automatically — define the *what*, not the *when*.

Data Quality with Expectations

DLT's Expectations system lets you encode business rules directly into your pipeline definition. This shifts data quality from an afterthought to a first-class citizen of your data platform.

```
# Warn only — log violation, keep record
@dlt.expect("non_negative_amount", "amount >= 0")

# Drop invalid records silently
@dlt.expect_or_drop("valid_customer", "customer_id IS NOT NULL")

# Fail pipeline on any violation
@dlt.expect_or_fail("critical_field", "order_id IS NOT NULL")
```

Three Enforcement Modes

- **expect** — record violation metrics, continue processing
- **expect_or_drop** — silently discard invalid rows
- **expect_or_fail** — halt the pipeline, alerting on-call engineers

All violations are surfaced in the DLT pipeline UI with row-level counts per expectation.

Streaming and Batch: One Unified Pipeline

One of DLT's most powerful capabilities is its ability to handle both **streaming** and **batch** workloads within a single pipeline definition — no separate codebase required.

Streaming Mode

Use `dlt.read_stream()` to consume continuously from Kafka, Auto Loader, or Delta Change Data Feed. Ideal for low-latency, event-driven pipelines where freshness is critical.

Batch Mode

Use `dlt.read()` for scheduled, full or incremental batch reads. DLT manages checkpointing and watermarking automatically — switching modes is a one-line change.

Unified Logic

The same transformation functions, quality expectations, and lineage tracking apply regardless of processing mode. Migrate from batch to streaming without rewriting your pipeline.

DLT vs Traditional ETL

DLT doesn't just modernise the toolchain — it fundamentally changes how engineering teams operate, shifting effort from infrastructure management to business logic.

Feature	Traditional ETL	Delta Live Tables
Orchestration	Manual DAG wiring (Airflow, cron)	Automatic dependency resolution
Data Quality	Custom validation scripts or none	Built-in Expectations framework
Incremental Loads	Custom bookmarking logic	Automatic via Delta checkpointing
Streaming Support	Separate codebase required	Unified batch + streaming API
Lineage & Observability	Manual documentation	Built-in lineage graph in UI
Error Handling	Custom retry logic per job	Managed retries and failure modes
Maintenance Overhead	High — distributed across tools	Low — centralised in DLT

Now, Let's Build It

Time to build a Medallion pipeline from scratch.

We've covered the theory — the problems DLT solves, how its declarative model works, and the power of built-in quality enforcement across streaming and batch workloads. Now it's time to put it into practice.

01

Define Bronze Layer

Ingest raw data from a source using Auto Loader with `@dlt.table`.

02

Add Silver Transformations

Clean, cast, and validate data with Expectations applied.

03

Aggregate to Gold

Build business-level aggregations ready for BI and ML consumption.

👉 Open your Databricks workspace — we're building this live, step by step.